

CSC455 - Computer Vision



Name: Aoun Haider

ID: FA21-BSE-133

Assignment: 02

Question 1:

Bloom Taxonomy Level: <Applying>

```
from PIL import Image
```

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from google.colab.patches import cv2_imshow
```

```
def add_gaussian_noise(image, mean=0, std=25):
```

```
    noise = np.random.normal(mean, std, image.shape).astype(np.uint8)
```

```
    noisy_image = cv2.add(image, noise)
```

```
    return noisy_image
```

```
def remove_noise(image, kernel_size=5):
```

```
    return cv2.medianBlur(image, kernel_size)
```

```
def adaptive_thresholding(image, grid_size):
```

```
    # Convert to HSV color space
```

```
    hsv = cv2.cvtColor(cv2.cvtColor(image, cv2.COLOR_GRAY2BGR), cv2.COLOR_BGR2HSV)
```

```
    # Create a mask for non-white pixels
```

```
    lower = np.array([0, 0, 0])
```

```
    upper = np.array([180, 255, 230])
```

```
mask = cv2.inRange(hsv, lower, upper)
```

```
# Apply morphological operations to clean up the mask
```

```
kernel = np.ones((5,5), np.uint8)
```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)
```

```
# Fill holes
```

```
h, w = mask.shape[:2]
```

```
mask_floodfill = mask.copy()
```

```
cv2.floodFill(mask_floodfill, np.zeros((h+2, w+2), np.uint8), (0,0), 255)
```

```
mask_floodfill_inv = cv2.bitwise_not(mask_floodfill)
```

```
mask_filled = mask | mask_floodfill_inv
```

```
# Invert colors to make objects black
```

```
result = cv2.bitwise_not(mask_filled)
```

```
# Calculate histograms
```

```
histograms = []
```

```
cell_height, cell_width = image.shape[0] // grid_size[0], image.shape[1] // grid_size[1]
```

```
for i in range(grid_size[0]):
```

```
    for j in range(grid_size[1]):
```

```
        y1, y2 = i * cell_height, (i + 1) * cell_height
```

```
        x1, x2 = j * cell_width, (j + 1) * cell_width
```

```
        cell = image[y1:y2, x1:x2]
```

```
        hist = cv2.calcHist([cell], [0], None, [256], [0, 256])
```

```
histograms.append(hist)
```

```
return result, histograms
```

```
def plot_histograms(histograms, grid_size):
```

```
    fig, axes = plt.subplots(grid_size[0], grid_size[1], figsize=(20, 20))
```

```
    axes = axes.ravel()
```

```
    for i, hist in enumerate(histograms):
```

```
        axes[i].plot(hist)
```

```
        axes[i].set_title(f'Cell {i+1}')
```

```
        axes[i].set_xlim([0, 256])
```

```
        axes[i].set_ylim([0, hist.max()])
```

```
plt.tight_layout()
```

```
plt.show()
```

```
def display_images(images, titles):
```

```
    fig, axes = plt.subplots(2, 2, figsize=(12, 12))
```

```
    axes = axes.ravel()
```

```
    for i, (image, title) in enumerate(zip(images, titles)):
```

```
        axes[i].imshow(image, cmap='gray')
```

```
        axes[i].set_title(title)
```

```
        axes[i].axis('off')
```

```
plt.tight_layout()

plt.show()

def main(image_path, grid_size):

    # Read the image in color

    image = cv2.imread(image_path)

    # Convert to grayscale for noise addition and removal

    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Add Gaussian noise

    noisy_image = add_gaussian_noise(gray_image)

    # Remove noise using median filter

    denoised_image = remove_noise(noisy_image)

    # Apply adaptive thresholding with improved edge detection

    thresholded, histograms = adaptive_thresholding(denoised_image, grid_size)

    # Display results

    display_images([gray_image, noisy_image, denoised_image, thresholded],

                    ['Original Image', 'Noisy Image', 'Denoised Image (Median Filter)', 'Thresholded Image'])

    if isinstance(thresholded, np.ndarray):

        pil_image = Image.fromarray(thresholded)
```

```
else:
```

```
    pil_image = thresholded
```

```
    pil_image.save("image_downloaded.png", format="PNG")
```

```
    # Plot histograms
```

```
    plot_histograms(histograms, grid_size)
```

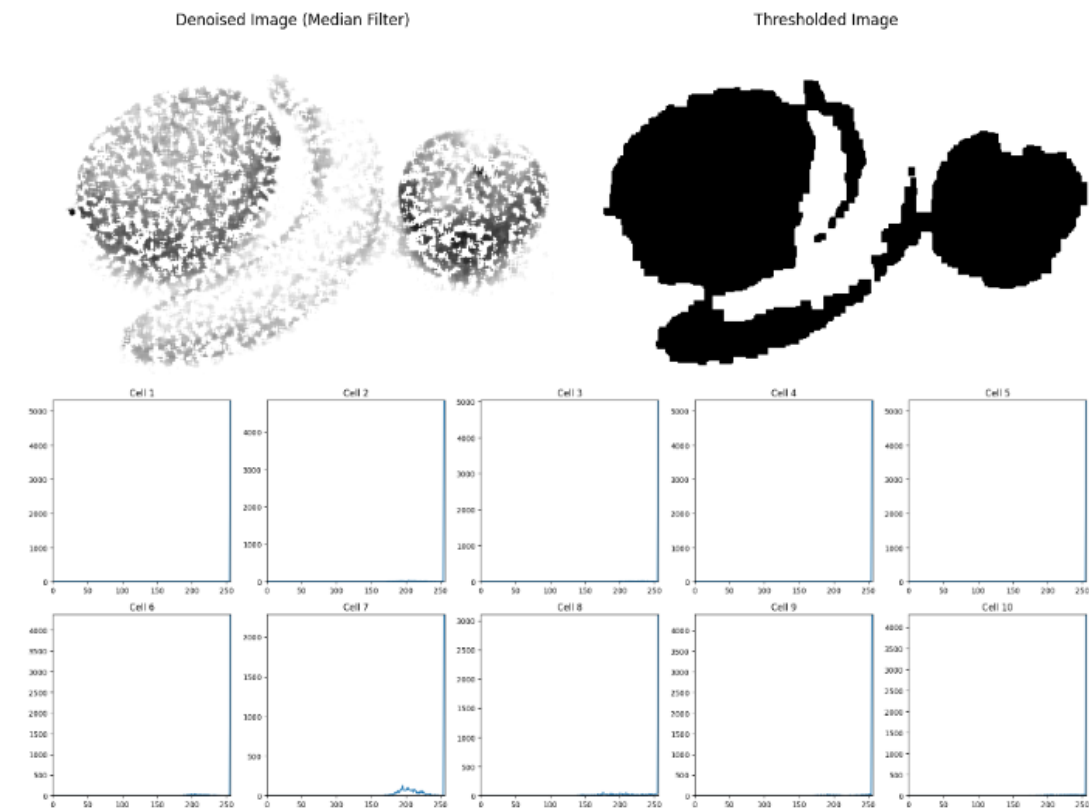
```
if __name__ == "__main__":
```

```
    image_path = "/content/img_1.png" # Replace with your image path
```

```
    grid_size = (5, 5) # Changed to 5x5 for better results with this method
```

```
    main(image_path, grid_size)
```

Output:



Question 2: (i)

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from google.colab.patches import cv2_imshow


def add_gaussian_noise(image, mean=0, std=25):

    noise = np.random.normal(mean, std, image.shape).astype(np.uint8)

    noisy_image = cv2.add(image, noise)

    return noisy_image


def remove_noise(image, kernel_size=5):

    return cv2.medianBlur(image, kernel_size)


def adaptive_thresholding(image, grid_size):

    height, width = image.shape

    cell_height, cell_width = height // grid_size[0], width // grid_size[1]


    # Apply adaptive thresholding

    thresholded = cv2.adaptiveThreshold(image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV, 11, 2)


    # Apply morphological operations to remove noise

    kernel = np.ones((3,3), np.uint8)

    thresholded = cv2.morphologyEx(thresholded, cv2.MORPH_CLOSE, kernel, iterations=2)

    thresholded = cv2.morphologyEx(thresholded, cv2.MORPH_OPEN, kernel, iterations=1)
```

```

# Connected component analysis to remove small noise

num_labels, labels, stats, centroids =
cv2.connectedComponentsWithStats(thresholded, connectivity=8)

# Remove small components (noise)

min_size = 100 # Adjust this value to control noise removal

for i in range(1, num_labels):

    if stats[i, cv2.CC_STAT_AREA] < min_size:

        thresholded[labels == i] = 0

# Invert the image to make objects black and background white

thresholded = cv2.bitwise_not(thresholded)

return thresholded, labels

def compute_area(labels, label):

    return np.sum(labels == label)

def compute_centroid(labels, label):

    y, x = np.where(labels == label)

    return np.mean(x), np.mean(y)

def compute_perimeter(labels, label):

    object_mask = (labels == label).astype(np.uint8)

    contours, _ = cv2.findContours(object_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_NONE)

```

```
return len(contours[0])
```

```
def compute_circularity(area, perimeter):  
    return (4 * np.pi * area) / (perimeter ** 2)
```

```
def compute_region_properties(labels):  
    properties = []  
    for label in range(1, labels.max() + 1):  
        area = compute_area(labels, label)  
        centroid = compute_centroid(labels, label)  
        perimeter = compute_perimeter(labels, label)  
        circularity = compute_circularity(area, perimeter)  
  
        properties.append({  
            'label': label,  
            'area': area,  
            'centroid': centroid,  
            'perimeter': perimeter,  
            'circularity': circularity  
        })
```

```
return properties
```

```
def display_images(images, titles):  
    fig, axes = plt.subplots(2, 2, figsize=(12, 12))  
    axes = axes.ravel()
```



```
for i, (image, title) in enumerate(zip(images, titles)):
```

```
    axes[i].imshow(image, cmap='gray')
```

```
    axes[i].set_title(title)
```

```
    axes[i].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
def main(image_path):
```

```
    # Read the image
```

```
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```
    # Add Gaussian noise
```

```
    noisy_image = add_gaussian_noise(image)
```

```
    # Remove noise using median filter
```

```
    denoised_image = remove_noise(noisy_image)
```

```
    # Apply adaptive thresholding
```

```
    thresholded, labels = adaptive_thresholding(denoised_image, (10, 10))
```

```
    # Compute region properties
```

```
    properties = compute_region_properties(labels)
```

```
    # Display results
```

```

display_images([image, noisy_image, denoised_image, thresholded],
               ['Original Image', 'Noisy Image', 'Denoised Image', 'Thresholded Image'])

# Print region properties
for prop in properties:
    print(f"Object {prop['label']}:")
    print(f" Area: {prop['area']}")
    print(f" Centroid: {prop['centroid']}")
    print(f" Perimeter: {prop['perimeter']}")
    print(f" Circularity: {prop['circularity']:.4f}")
    print()

if __name__ == "__main__":
    image_path = "/content/image_downloaded.png" # Replace with your image path
    main(image_path)

```

Output:

```

Object 1:
Area: 1037
Centroid: (234.01350048216008, 80.35872709739634)
Perimeter: 266
Circularity: 0.1842

Object 2:
Area: 23892
Centroid: (132.3151682571572, 155.38858195211787)
Perimeter: 1357
Circularity: 0.1630

Object 3:
Area: 12112
Centroid: (369.5624174372523, 156.21846103038308)
Perimeter: 768
Circularity: 0.2580

Object 4:
Area: 9
Centroid: (298.0, 120.0)
Perimeter: 8
Circularity: 1.7671

```

Question 2: (ii)

```
from PIL import Image, ImageDraw, ImageFont
```

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def label_fruits(image_path, output_path):
```

```
    # Open the image
```

```
    img = Image.open(image_path)
```

```
    draw = ImageDraw.Draw(img)
```

```
    # Load a font
```

```
    try:
```

```
        font = ImageFont.truetype("arial.ttf", 36)
```

```
    except IOError:
```

```
        font = ImageFont.load_default()
```

```
    # Define fruits and their positions
```

```
    fruits = [
```

```
        ("Mango", (50, 300)),
```

```
        ("Banana", (250, 300)),
```

```
        ("Apple", (450, 300))
```

```
    ]
```

```
    # Add labels
```

```
    for fruit, position in fruits:
```

```
draw.text(position, fruit, fill=(255, 255, 0), font=font)
```

```
# Save the labeled image
```

```
img.save(output_path)
```

```
def preprocess_image(image_path):
```

```
    image = cv2.imread(image_path)
```

```
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
    blurred_image = cv2.GaussianBlur(image, (5, 5), 0)
```

```
    hsv_image = cv2.cvtColor(blurred_image, cv2.COLOR_RGB2HSV)
```

```
    return image, hsv_image
```

```
def create_mask(hsv_image, lower, upper):
```

```
    mask = cv2.inRange(hsv_image, lower, upper)
```

```
    kernel = np.ones((5,5), np.uint8)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
```

```
    return mask
```

```
def classify_fruits(original_image, hsv_image):
```

```
    # Define color ranges for each fruit in HSV
```

```
    banana_lower1 = np.array([20, 100, 100])
```

```
    banana_upper1 = np.array([30, 255, 255])
```

```
    banana_lower2 = np.array([30, 50, 200])
```

```
    banana_upper2 = np.array([40, 100, 255])
```

```
mango_lower = np.array([20, 50, 50])
mango_upper = np.array([60, 255, 255])
```

```
apple_lower = np.array([0, 100, 100])
apple_upper = np.array([10, 255, 255])
```

```
# Create masks for each fruit
```

```
banana_mask1 = create_mask(hsv_image, banana_lower1, banana_upper1)
banana_mask2 = create_mask(hsv_image, banana_lower2, banana_upper2)
banana_mask = cv2.bitwise_or(banana_mask1, banana_mask2)
mango_mask = create_mask(hsv_image, mango_lower, mango_upper)
apple_mask = create_mask(hsv_image, apple_lower, apple_upper)
```

```
# Combine masks
```

```
combined_mask = banana_mask | mango_mask | apple_mask
```

```
# Find contours for the combined mask
```

```
contours, _ = cv2.findContours(combined_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

```
for contour in contours:
```

```
    area = cv2.contourArea(contour)
```

```
    if area > 5000: # Adjust this threshold as needed
```

```
        x, y, w, h = cv2.boundingRect(contour)
```

```
        roi = hsv_image[y:y+h, x:x+w]
```

```

# Calculate the percentage of pixels that match each fruit's color range
banana_percent = np.sum(create_mask(roi, banana_lower2, banana_upper2)) / (w *
h)

mango_percent = np.sum(create_mask(roi, mango_lower, mango_upper)) / (w * h)
apple_percent = np.sum(create_mask(roi, apple_lower, apple_upper)) / (w * h)


# Classify based on the highest percentage
if banana_percent > mango_percent and banana_percent > apple_percent:
    label = "Banana"
    color = (255, 255, 0) # Yellow in RGB
elif mango_percent > apple_percent:
    label = "Mango"
    color = (0, 255, 0) # Green in RGB
else:
    label = "Apple"
    color = (255, 0, 0) # Red in RGB


cv2.rectangle(original_image, (x, y), (x + w, y + h), color, 2)
cv2.putText(original_image, label, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9,
color, 2)


return original_image


def main(image_path):
    original_image, hsv_image = preprocess_image(image_path)
    classified_image = classify_fruits(original_image, hsv_image)

```

```
plt.figure(figsize=(10, 10))  
plt.imshow(classified_image)  
plt.title("Fruits Classified and Labeled")  
plt.axis("off")  
plt.show()
```

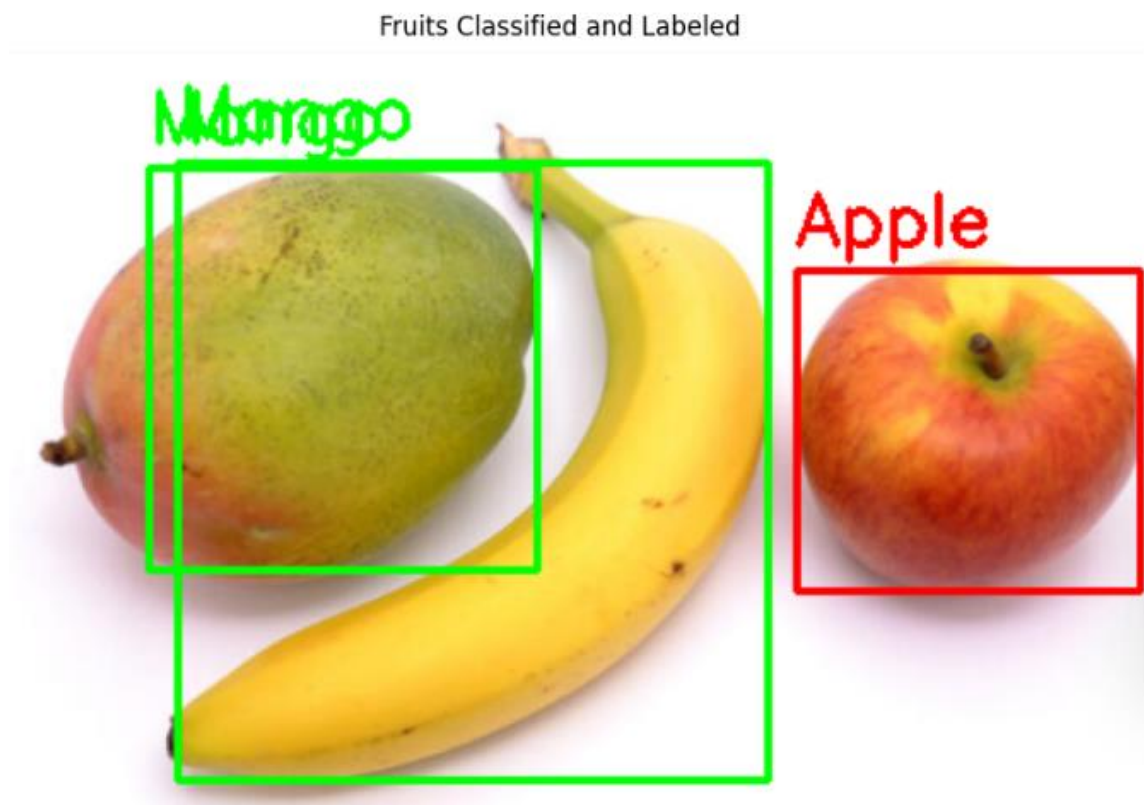
Example usage

```
image_path = "img_1.png" # Update this to your image path  
main(image_path)
```

Use the function

```
label_fruits("img_1.png", "part_b_2.png")
```

Output:



Question 2: (iii)

```
from PIL import Image, ImageDraw, ImageFont
```

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from rembg import remove
```

```
import io
```

```
def label_fruits(image_path, output_path):
```

```
    # Open the image
```

```
    img = Image.open(image_path)
```

```
    draw = ImageDraw.Draw(img)
```

```
    # Load a font
```

```
    try:
```

```
        font = ImageFont.truetype("arial.ttf", 36)
```

```
    except IOError:
```

```
        font = ImageFont.load_default()
```

```
    # Define fruits and their positions
```

```
    fruits = [
```

```
        ("Mango", (50, 300)),
```

```
        ("Banana", (250, 300)),
```

```
        ("Apple", (450, 300))
```

```
    ]
```



```
# Add labels
```

```
for fruit, position in fruits:
```

```
    draw.text(position, fruit, fill=(255, 255, 0), font=font)
```

```
# Save the labeled image
```

```
img.save(output_path)
```

```
def preprocess_image(image_path):
```

```
    image = cv2.imread(image_path)
```

```
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
    return image
```

```
def remove_background(image_path):
```

```
    with open(image_path, "rb") as input_file:
```

```
        input_bytes = input_file.read()
```

```
    output_bytes = remove(input_bytes)
```

```
    output_image = Image.open(io.BytesIO(output_bytes))
```

```
    output_array = np.array(output_image)
```

```
# Create a mask from the alpha channel
```

```
mask = output_array[:, :, 3]
```

```
# Apply morphological operations to increase gaps between fruits
```

```
kernel = np.ones((7,7), np.uint8)
```

```
eroded_mask = cv2.erode(mask, kernel, iterations=2)
```

```
dilated_mask = cv2.dilate(eroded_mask, kernel, iterations=2)
```

```
# Apply the modified mask

rgb_image = cv2.cvtColor(output_array, cv2.COLOR_RGBA2RGB)

result = np.zeros_like(rgb_image)

for c in range(3):

    result[:, :, c] = rgb_image[:, :, c] * (dilated_mask / 255)


# Set background to black

result[dilated_mask == 0] = [0, 0, 0]


return result
```

```
def classify_fruits(image):

    hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)


    # Define color ranges for oranges

    orange_lower = np.array([10, 100, 100])
    orange_upper = np.array([25, 255, 255])


    # Define color ranges for mango (red and green components)

    mango_red_lower = np.array([0, 50, 50])
    mango_red_upper = np.array([10, 255, 255])
    mango_green_lower = np.array([35, 50, 50])
    mango_green_upper = np.array([85, 255, 255])


    # Create masks
```

```

orange_mask = cv2.inRange(hsv, orange_lower, orange_upper)

mango_red_mask = cv2.inRange(hsv, mango_red_lower, mango_red_upper)

mango_green_mask = cv2.inRange(hsv, mango_green_lower, mango_green_upper)

mango_mask = cv2.bitwise_or(mango_red_mask, mango_green_mask)


# Find contours

contours_orange, _ = cv2.findContours(orange_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

contours_mango, _ = cv2.findContours(mango_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)


labeled_image = image.copy()


# Label oranges

for contour in contours_orange:

    area = cv2.contourArea(contour)

    if area > 5000:

        x, y, w, h = cv2.boundingRect(contour)

        roi = hsv[y:y+h, x:x+w]

        orange_ratio = np.sum((roi[:, :, 0] >= 10) & (roi[:, :, 0] <= 25)) / (w * h)

        if orange_ratio > 0.5:

            cv2.rectangle(labeled_image, (x, y), (x + w, y + h), (255, 165, 0), 2)

            cv2.putText(labeled_image, "Orange", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.9, (255, 165, 0), 2)


# Label mango

for contour in contours_mango:

```

```

area = cv2.contourArea(contour)

if area > 10000:

    x, y, w, h = cv2.boundingRect(contour)

    roi = hsv[y:y+h, x:x+w]

    red_ratio = np.sum((roi[:, :, 0] >= 0) & (roi[:, :, 0] <= 10)) / (w * h)

    green_ratio = np.sum((roi[:, :, 0] >= 35) & (roi[:, :, 0] <= 85)) / (w * h)

    if red_ratio > 0.1 and green_ratio > 0.1:

        cv2.rectangle(labeled_image, (x, y), (x + w, y + h), (0, 255, 0), 2)

        cv2.putText(labeled_image, "Mango", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9,
(0, 255, 0), 2)

return labeled_image


def main(image_path):

    processed_image = remove_background(image_path)

    classified_image = classify_fruits(processed_image)


original_image = cv2.imread(image_path)

original_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)


plt.figure(figsize=(15, 5))

plt.subplot(131)

plt.imshow(original_image)

plt.title("Original Image")

plt.axis("off")

```

```
plt.subplot(132)
plt.imshow(processed_image)
plt.title("Processed Image (Background Removed)")
plt.axis("off")
```

```
plt.subplot(133)
plt.imshow(classified_image)
plt.title("Fruits Classified and Labeled")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Example usage

```
image_path = "img_2.png" # Update this to your image path
```

```
main(image_path)
```

Use the function

```
label_fruits("img_2.png", "part_b_3.png")
```

Output:

